

Mechanism for http Certificate & Installing php MyAdmin

Tom Goncin

July 15th, 2026

ENSE 353 – Lab 4

Objective (What did we do?):

Understanding and producing a secure Apache https web domain connection. This includes the manual generation, and self signing to deploy my own SSL/TLS connection, using public key cryptography. Once complete, configuring php as my administrative interface for MariaDB which involved solving SELinux context restriction.

Steps (How did we do it?)

Part 1: Standard Boot Procurement

1. Login as user through browser remotely
2. Login as root

```
su -
```

3. Upgrade

```
dnf install -refresh
```

4. Reboot

```
shutdown -r now
```

5. Show gateway address

```
ip route
```

```
142.3.24.1/23
```

Part 2: Configure Apache https

1. Login again to root once rebooted

```
su -
```

2. Install software package that provides a SSL/TLS module for the Apache HTTP web server

```
dnf install mod_ssl
```

3. Check

```
telnet 127.0.0.1 443
```

Should not have worked

```
systemctl restart httpd
```

4. Repeat step 3

5. Open Port for http and reload firewall

```
firewall-cmd --permanent --zone=public --add-service=https
```

6. Reload firewall

```
Firewall-cmd --reload
```

7. List all

```
firewall-cmd --list-all
```

8. Go to a browser and type in your web address

```
https://142.3.24.32
```

Part 3: Generation of Certificate Files

1. Load

```
openssl req -new -sha256 -newkey rsa:2048 -nodes -keyout my.key -out my.csr
```

2. Follow the requests

3. Self sign the certificate. It will cause webpages to trip

```
openssl x509 -req -days 365 -in my.csr -signkey my.key -out my.crt
```

4. Go to

```
Cd /etc/httpd/conf.d/
```

5. Open in text editor

```
Emacs ssl.conf
```

6. In the file, not too far down at 34%, you will see:

```
SSLCertificateKeyFile /etc/pki/tls/private/localhost.key
```

Change it to my.key file and move it into the directory

Just above it change the localhost.crt to my.crt

7. Go to certificate folder and do ls

```
/etc/pki/tls/certs
```

8. Move files to the appropriate spot

```
in that folder  
mv /root/my.crt .
```

9. Do it with the private key now

```
cd /etc/pki/tls/private  
mv root/my.key .
```

10. Restart Apache

```
systemctl restart httpd
```

Should not work. seLinux designed this to prevent security hazards, so security is enables on Linux Rocky

11. To see if it is seLinux, turn it off temporarily and you can see if it is that

```
setenforce 0
```

12. Make it okay with Apache

```
restorecon -rv /etc/tki/tls/certs/
```

This will restore the configurations for seLinux

Should have relabeled

13. Same for my.key

```
restorecon -rv /etc/tki/tls/private/
```

14. Turn on seLinux and do

```
setenforce 1
```

15. Restart Apache

```
systemctl restart httpd
```

It should work.

Part 4: Shift from administering database from MariaDB to phpAdmin

1. Go to default Apache directory

```
Cd /var/www/html
```

2. Install package for downloading files from online through links

```
Dnf install wget
```

3. Go to phpmyadmin.net

Go to Download

Download:

<https://files.phpmyadmin.net/phpMyAdmin/5.2.3/phpMyAdmin-5.2.3-english.tar.gz>

4. Install tar (tape archive)

```
Dnf install tar
```

5. Click ls and extract

```
Tar xvf phpMyAdmin-5.2.3-english.tar.gz
```

6. Move it into a folder named phpMyAdmin and go to webpage and type in <http://ipaddress/phpMyAdmin>

7. Make a random hex 32 char

```
Openssl rand -hex 32
```

8. Look in that file and edit the file

```
Emacs config.inc.php
```

9. At blowfish_secret in “ do around 6%, add the value you received, like this:

```
Sodium_hex2bin('#####32bits#####');
```

10. Make directory

```
Mkdir tmp
```

11. Give privileges to Apache

```
Chown apache tmp
```

12. Will not work, configure seLinux to allow this. Make sure its seLinux with

```
setenfore 0
```

13. We know that's it now lets

```
Semanage fcontext -a -t httpd_sys_rw_content_t /var/www/html/phpMyAdmin/tmp(/.*)?"
```

14. Restart after

```
Systemctl restart http`
```

Outcome (Why did we do it?)

HTTPS configured through the self-signing certificate to protect the server's sensitive data with administrative credentials and queries. The encryption was done through port 443, which protects our server against unwanted interceptions.

The phpAdmin tool was configured for database management which provides us, the user, a graphical interface.

Another tool/method provided was in identifying and surpassing the SELinux protective system with a compulsory MAC (Mandatory Access Control) along with other tools such as **restorecon** and **semanage fcontext** that allows us to codify Apache user access by properly labeling the file contexts that reduce the need to permanently disable our servers security system.